CV Screening Multi-Agent System - Project Report

1. Project Information

- Project title: CV Screening Multi-Agent System
- Course context: Integrated AI/ML software engineering project
- Project goal: Build an intelligent screening assistant that evaluates candidate CVs against job descriptions using multiple specialist agents, a deep learning model, and human-in-the-loop control.

2. Problem Statement

Recruiters and hiring teams receive many applications and need a fast, consistent way to evaluate candidate-job fit. Manual screening is time-consuming and can be inconsistent. The project addresses this by creating a multi-agent workflow that:

- Separates technical and profile analysis into specialized agents
- Uses a trained PyTorch model for technical scoring support
- Uses explainable intermediate outputs (matched skills, missing skills, rationales)
- Escalates uncertain cases to human review
- Logs decisions for traceability and auditability

3. Main Objectives

1. Design and implement a working multi-agent screening pipeline
2. Integrate a real deep learning model into agent decision flow
3. Include at least two practical tools used by agents
4. Add a human-in-the-loop mechanism for borderline/conflicting decisions
5. Build an interactive frontend for live presentation
6. Evaluate performance with metrics and test cases

4. System Architecture

4.1 Agents

- Technical Matcher Agent:
  - Focus: hard skills and technical fit
  - Uses model output and skill overlap
  - Produces score, label, rationale, matched/missing skills

- Profile Analyzer Agent:
  - Focus: seniority, communication, project ownership, education cues
  - Produces score, label, rationale

- Orchestrator Agent:
  - Combines technical and profile outputs
  - Applies weighted final score
  - Detects disagreement and borderline confidence
  - Triggers human review policy when needed

4.2 Tools

- Model Tool:

  - Loads trained PyTorch model
  - Produces probability/fit signal for technical assessment

- Skill Extractor Tool:
  - Extracts relevant skills from CV and job text
  - Computes matched skills, missing skills, and coverage

4.3 Human-in-the-Loop Policy

Human review is triggered if:

- Final score is in borderline range
- Technical and profile agents strongly disagree
- Any agent falls back due to failure

Strict HITL is available in both CLI and frontend:

- CLI strict mode (`--require-human-approval`) marks pending review when no interactive reviewer is available.
- Streamlit Single Screening mode includes a blocking HR decision step (Shortlist/Reject/Needs Review) before final recommendation is finalized.

5. Data and Model

5.1 Data

- Sample CV data: multiple candidate profiles
- Sample job data: four built-in roles: Junior Data Analyst, Business Intelligence Analyst, Machine Learning Intern, and Data Engineering Assistant
- Labeled evaluation dataset for pipeline validation

5.2 Model

- Framework: PyTorch
- Artifact: models/cv_fit_model.pt
- Baseline evaluation artifact: models/model_evaluation.json

5.3 Model Metrics (from artifact)

- Accuracy: 1.0
- Macro F1: 1.0
- Per-class precision/recall/F1: all 1.0 on evaluation split
- Samples evaluated: 60

6. Pipeline and Decision Logic

1. Parse input CV and job description
2. Technical agent computes technical fit score and rationale
3. Profile agent computes profile fit score and rationale
4. Orchestrator computes weighted final score
5. Final label mapping:
   - High
   - Medium

   - Low
6. Recommendation mapping:
   - High -> shortlist
   - Medium -> review
   - Low -> reject
7. Optional LLM enrichment (Ollama) generates short professional recommendation text
8. Log all steps and outputs to JSONL

6.1 Agent Framework Activation (CrewAI Primary)

- Primary runtime backend: CrewAI in `auto` mode when available
- Fallback runtime backend: deterministic orchestrator for resilience
- CrewAI activation environment: Python 3.12 virtual environment (`.venv312`)
- Activation script: `setup_crewai_primary.ps1`
- Runtime command: `.venv312\\Scripts\\python.exe -m src.main --runtime auto --ollama`

6.2 LangGraph Stretch Goal

- Stretch-goal scaffold included in `src/langgraph_runtime.py`
- Current scope: minimal compiled graph foundation for future branching/retry workflows
- Status: integrated as optional extension, not required for core grading criteria

7. Frontend Features (Presentation Focus)

The Streamlit frontend focuses on two demo modes that match the simplified HR use case:

1. Single Screening
   - One CV vs one job
   - Final label, score, recommendation
   - Agent outputs and rationale
   - Skill gap panel
   - Explainability highlights

2. Batch Screening
   - Multiple CVs vs one selected job
   - Leaderboard ranking
- Visible candidate analysis cards with agent rationales, matched skills, missing skills, and review reasons
   - Inline HR review decision capture for flagged or pending candidates
   - Charts and downloadable reports

Additional presentation capabilities:

- PDF CV upload and text extraction
- Session history panel
- JSON and CSV export
- Defense checklist panel showing brief alignment evidence
- Showcase metrics panel from model and pipeline artifacts
- Ollama server/model controls in sidebar

8. Ollama Integration

Ollama was integrated as optional local LLM backend for enriched recommendations:

- Endpoint: http://localhost:11434
- Default model: llama3.2
- Added runner script to ensure environment readiness before launch

Project runner script:

- run_project.ps1
- Ensures dependencies
- Ensures Ollama is running
- Ensures model availability
- Launches Streamlit app

9. Evaluation and Testing

9.1 Automated Tests

- Test framework: pytest
- Status: all tests passing (16/16)

9.2 Pipeline Evaluation

- Artifact: logs/pipeline_evaluation.json
- Cases: 6 labeled screening cases
- Reported accuracy: 0.8333
- Includes case-level predictions and review reasons

9.3 Traceability

- Runtime logs stored in logs/run_*.jsonl
- Supports debugging and decision audit trail

Structured logging fields included in each event:

- timestamp
- agent_name
- action
- tool_used
- input_summary
- output_summary
- status
- error
- event
- payload

10. Alignment With Assignment Requirements

Requirement coverage summary:

1. Multi-agent architecture: completed
2. Agent framework (CrewAI primary): completed (Python 3.12 runtime)
3. Deep learning model usage: completed
4. Tools integration: completed

5. Human-in-the-loop checkpoint: completed
6. Working user interface: completed
7. Evaluation and testing evidence: completed
8. Reproducibility (setup and run instructions): completed

11. Challenges and Solutions

Challenge 1: Long response time with Ollama

- Cause: LLM generation latency and occasional startup delay
- Solution:
  - Added local server checks
  - Added model readiness checks
  - Reduced generated token length for faster response

Challenge 2: Streamlit waiting on CLI human input

- Cause: borderline cases triggered terminal prompt in web flow
- Solution:
  - Added strict human approval mode in frontend Single Screening flow
  - Added explicit reviewer decision capture (Shortlist/Reject/Needs Review)
  - Added visible batch review controls for pending/flagged candidates

Challenge 3: Port conflicts during demo startup

- Cause: stale Streamlit processes occupying ports
- Solution:
  - Added controlled startup flow and alternative port launch behavior

12. Limitations

- Current model is trained on project dataset and should be further validated on larger real-world data.
- Skill extraction is keyword-oriented and can miss deeper semantic equivalence.
- Ollama quality and speed depend on local hardware resources.
- CrewAI availability depends on using a compatible local runtime (Python 3.11/3.12 recommended).

13. Future Improvements

1. Add semantic embedding-based skill matching
2. Add richer fairness and bias analysis metrics
3. Add recruiter feedback loop for continuous learning
4. Add dashboard analytics for trend monitoring
5. Add multilingual CV/job support

14. How to Reproduce

1. Create and activate virtual environment
2. Install dependencies from requirements.txt
3. Run training and evaluation if needed
4. Launch app with runner script for smooth demo:

PowerShell command:

powershell -ExecutionPolicy Bypass -File .\run_project.ps1 -SkipDependencyInstall -Port 8501

Strict HITL CLI command:

```powershell
.venv312\Scripts\python.exe -m src.main --runtime deterministic --require-human-approval
```

15. Recommended Word Report Structure

Your classmates can convert this Markdown into a Word report with these sections:

1. Abstract
2. Introduction and Problem Statement
3. System Architecture
4. Methodology and Tools
5. Model and Evaluation
6. Frontend Demonstration
7. Results and Discussion
8. Challenges and Fixes
9. Conclusion and Future Work
10. Appendix (screenshots, logs, outputs)

16. Suggested Figures/Tables for Word Version

- Architecture diagram (agents + tools + orchestrator)
- Single screening result screenshot
- Batch leaderboard screenshot
- Model metrics table
- Pipeline case evaluation table

17. Conclusion

This project successfully implements a practical CV screening platform that combines multi-agent reasoning, deep learning, explainability, and human governance. It is presentation-ready, test-validated, and aligned with academic project requirements.

Appendix A - Reproducibility Checklist

1. Python version and environment details
- Recommended local runtime: Python 3.11/3.12 for CrewAI path.
- Deterministic runtime is available for wider compatibility.

2. Setup commands
- pip install -r requirements.txt
- python -m src.train_baseline
- python -m src.main --runtime auto
- python -m src.evaluate_pipeline

3. Strict HITL validation
- python -m src.main --runtime deterministic --require-human-approval

- Expected behavior: pending review in non-interactive contexts and explicit review capture in frontend single mode.

4. Output artifacts
- models/cv_fit_model.pt
- models/model_evaluation.json
- logs/pipeline_evaluation.json
- logs/run_*.jsonl

Appendix B - Logging Contract

Each runtime log event stores the following fields:
- timestamp
- agent_name
- action
- tool_used
- input_summary
- output_summary
- status
- error
- event
- payload

This schema enables consistent debugging, failure triage, and oral-defense traceability.

Appendix C - Guardrails and Failure Behavior

Guardrails:
- Borderline score threshold policy
- Specialist disagreement threshold
- Human checkpoint for sensitive decisions
- Fallback behavior for tool and LLM failures

Failure handling examples:
- Invalid JSON/file type in CLI: graceful error + hint output
- Missing model or load error: deterministic fallback prediction
- Ollama failure: runtime warning and non-crashing continuation

Appendix D - Defense Notes

Key design rationale to defend:
- Why technical and profile signals are separated into specialists
- Why orchestrator weighting and review thresholds exist
- Why strict HITL is needed for high-impact screening decisions
- Why structured JSON logging is essential for auditability

Known limitations and roadmap:
- Dataset scale and realism expansion
- Semantic matching and fairness diagnostics
- Batch-level reviewer workflow enhancements

Appendix E - Example Defense Q and A

Q1: Why separate technical and profile analysis?
A1: Separation improves interpretability, enables targeted evidence, and avoids collapsing heterogeneous signals into one opaque score.

Q2: Why does the orchestrator keep weighted scoring?
A2: Weighted aggregation provides deterministic governance and can be defended with explicit trade-offs between technical fit and profile fit.

Q3: Why require HITL for borderline decisions?
A3: Borderline and conflict cases have higher decision uncertainty; a reviewer checkpoint prevents automated overreach.

Q4: How is model integration meaningful?
A4: The technical specialist directly uses model output in scoring, and model confidence/evidence is propagated to the final decision context.

Q5: How are failures prevented from crashing the pipeline?
A5: Tool-level try/except guards, fallback outputs, and orchestrator review escalation keep runs resilient.

Q6: What proves reproducibility?
A6: Stable commands for setup, training, evaluation, runtime execution, and persistent JSONL traces for replay and audit.

Q7: Why keep CrewAI with deterministic fallback?
A7: CrewAI fulfills framework requirements while fallback preserves continuity in environments where optional dependencies are unavailable.

Q8: What are next engineering priorities?
A8: Larger datasets, semantic retrieval for skill matching, stricter schema validation, and enhanced reviewer workflow analytics.